

Fig. 3 Two-layer tamper-evidence on the audit hash chain

Left: intact live DB. Right: after a forced edit to a signed approval.

INTACT

```
db verify-chain
```

```
PASS      events
           scope 44599e2ec0f4... n=12
PASS      events
           scope 45a0644109ef... n=30
PASS      release_approvals
           scope 44599e2ec0f4... n=4
PASS      release_approvals
           scope 45a0644109ef... n=3
```

all chains intact

```
exit 0
```

AFTER EDIT

```
db verify-chain
```

```
PASS      events
           scope 44599e2ec0f4... n=12
PASS      events
           scope 45a0644109ef... n=30
PASS      release_approvals
           scope 44599e2ec0f4... n=4
FAIL      release_approvals
           scope 45a0644109ef... n=3 first_bad_seq=2
```

TAMPER DETECTED

```
exit 1
```

Two layers of defence:

- ① Append-only DB triggers on events/tool_calls/decision_traces— a plain UPDATE is rejected: “events is append-only (audit hash-chain)”
- ② The per-scope hash chain covers all four tables, including release_approvals (trigger-free). A direct edit to a signed approval leaves row_hash stale, so verify-chain flags the exact row.